the size of the array when processing it. Also, the related numbers have to be used, like 10-1, 10+1, etc.

If the size of the array has to be changed for some reason, changing the number '10' to some other number has to be done in multiple places. This introduces a scope for error, as there is a chance that one or more occurrences of the number may be missed.

Defining the list size using *#define* will avoid this problem, as shown below:

```
#define SIZE 10
```

The magic number is avoided in the code. Any change in the number can also be implemented with a change in a single place, which takes effect throughout the code.

Constant on the left hand side

When checking for equality, one of the "=" signs is inadvertently left out and the assignment operator "=" is typed instead of "==", the latter being the relational operator to test equality in a comparison statement. But it is not an error as per the C compiler and is treated as an assignment statement. Hence, the *if* statement returns true as long as the value assigned is not zero, and this bug is difficult to detect.

```
If (i = 10)        /* beg your pardon for using a magic
number! */
{
    ….
    ….
}
```

The above lines assign 10 to the variable 'i' and then test whether the value of *i* is true (non-zero) or false (0). Since the value of *i* always results in 10, the result of the Boolean expression will always be true.

The best way to prevent such bugs in the program is to write the constant on the left hand side. The compiler will flag an error if "=" is written instead of "==" since a constant cannot be assigned a value.

```
If (10 == i)
{
    ….
    ….
}
```

Naming of variables

Care should be taken when naming variables. It is better to use *i*, *j*, *k*, *m* and *n* as names of integer variables; *x*, *y* and *z* for float (or double) variables; and *a*, *b* and *c* as names of arrays or coefficients so as to make the program easy to read and to reduce the effort to understand the code.

Avoid using similar looking characters like *1* and *l* as these could be interpreted wrongly by the reader[1].

Beginners can go through the best practices for the naming of variables and functions, and follow a particular standard, consistently.

Avoid using the *gets* function

Many beginners use the *gets* function to input a string. Since it does not have a way to limit the size of the input string, input data can be bigger than the allocated space for the input, which will result in buffer overflow. The *gets* function has been deprecated in the C99 standard, and has been removed from the latest C11 standard. I am writing about the *gets* function as I have seen Turbo C being used even by MCA students in a few colleges in India!

The *fgets* function can be used to avoid the above mentioned problem, as the maximum length of the characters that can be read is given as an argument to the function.

```
#define MAX 20
….
char name[MAX];
…
…
fgets (name, MAX, stdin);
…
```

In the above code, *stdin* is the input stream.

Commenting out blocks of code

The C language does not allow nesting of comments. Hence, the /* and */ symbols cannot be used to comment out sections of the code temporarily for testing purposes.

Fortunately, the *#if... # endif* preprocessor directive can be used to leave out blocks of code from being compiled.

```
#if 0
……..
……..
#endif
```

The above method makes it easy to comment out large sections of code temporarily.

Reference

[1] *Introduction to computer science- An algorithmic approach by Tremblay and Bunt. McGraw- Hill international edition*

By: S. Sathyanarayanan

The author works as information scientist in the Sri Sathya Sai Institute of Higher Learning, Brindavan Campus, Bengaluru, and also heads the computer centre of the campus. He has more than 25 years of experience in systems administration and in teaching IT courses. He is an enthusiastic promoter of FOSS and can be reached at *sathyanarayanan_s@yahoo.com/* and *ssathyanarayanan@sssihl.edu.in.*